基于深度学习的葡萄酒质量评分实践



本文档不可复制，下载等，如需要可去：基于深度学习的葡萄酒质量评分实践(可下载复制)

0、代码说明

压缩包下载地址：基于深度学习的葡萄酒质量评分实践(可下载复制)

此处为动图（内容见飞书原文）

```
# 划分数据集：
python3 split_dataset.py

# 模型训练
```

```
python3 main.py

# 超参数
python3 main_opt.py

# 更换数据集后训练
python3 main_new_dataset.py
```

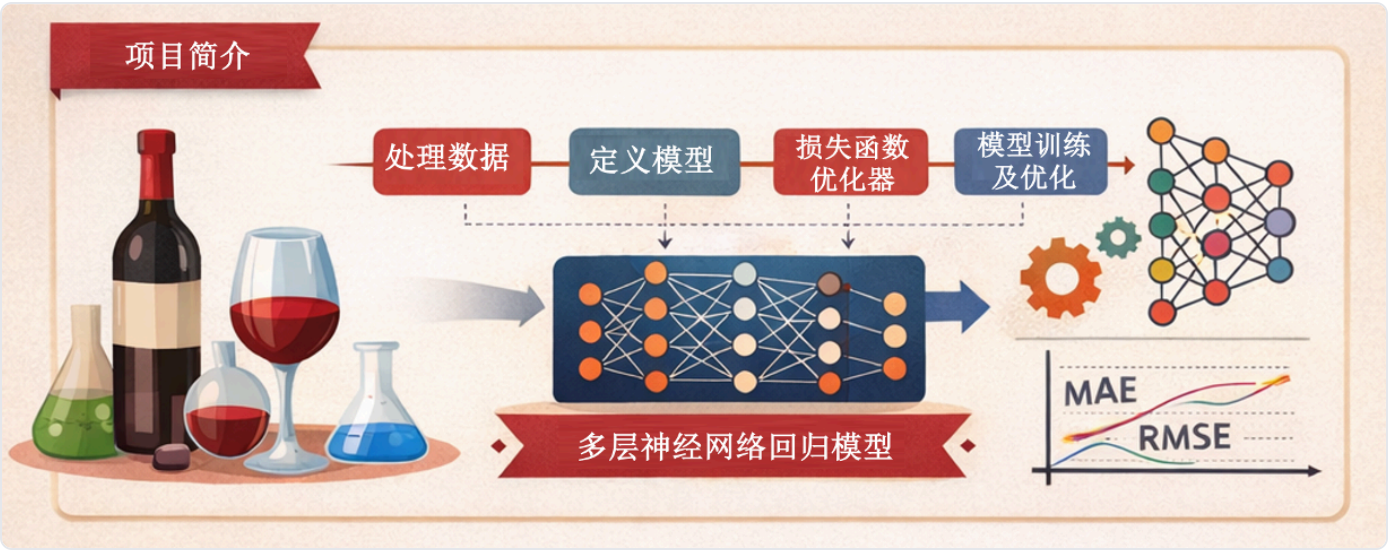
一、项目讲解

> 对应刘二大人，p7 处理多维度特征输入和p8 加载数据集

1.1 、项目简介

1. 问题定义

葡萄酒质量预测是一个**监督学习** 问题，其核心目标是：**根据葡萄酒的理化属性（ph值、酸度等），预测其感官质量评分。**



2.数据集介绍

a 数据来源与背景

Wine Quality 是来自 **UCI Machine Learning Repository** 的公开数据集，最初由 **Paulo Cortez 等人（2009）** 提出，用于研究葡萄酒理化指标与感官评分之间的关系，并发表在 *Decision Support Systems* 上。(UCI机器学习库)

数据可从 **UCI Machine Learning Repository** 官方页面下载：*Wine Quality Dataset* — https://archive.ics.uci.edu/dataset/186/wine+quality (UCI机器学习库)

- **采样地**：葡萄牙北部*Vinho Verde*葡萄酒产区。
- **葡萄酒类型**：包含 **红葡萄酒（red wine）** 与 **白葡萄酒（white wine）** 两部分。
- **采样方法**：理化属性通过实验室分析获得；质量评分由品酒专家通过感官评估给出。
- **隐私说明**：由于隐私和物流原因，**只提供理化输入变量及感官输出评分**（无品牌、价格、葡萄品种等信息）。

该数据集可以用于**监督回归或分类任务**，且目标标签分布呈现**类不平衡**（中等葡萄酒比极好或极差更多）。

---

b 数据集大小

- 我们只研究白葡萄酒，数据集一共4898条。
- 训练集行数: 3446 (70.4%)
- 验证集行数: 752 (15.4%)
- 测试集行数: 700 (14.3%)

3. 输入特征

每个样本包含 **11 个连续型数值特征**，例如：1、fixed acidity（固定酸度）2、volatile acidity（挥发性酸度）3、citric acid（柠檬酸）4、residual sugar（残糖）5、chlorides（氯化物）6、free sulfur dioxide（游离二氧化硫）7、total sulfur dioxide（总二氧化硫）8、density（密度）9、pH 10、sulphates（硫酸盐）11、alcohol（酒精度）

这些特征均来源于**实验室理化检测**，具有明确的物理意义。



固定酸度	挥发酸度	柠檬酸	残糖	氯化物	游离二氧化硫	总二氧化硫	密度	pH 值	硫酸盐	酒精度	品质
7.4	0.7	0.0	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
9.2	0.52	1.0	3.4	0.61	32.0	69.0	0.9996	2.74	2.0	9.4	4
7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.997	3.26	0.65	9.8	5
11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.998	3.16	0.58	9.8	6
10.4	0.61	0.49	2.1	0.2	5.0	16.0	0.9994	3.16	0.63	8.4	3
7.4	0.66	0.0	1.8	0.075	13.0	40.0	0.9978	3.51	0.56	9.4	5
7.9	0.6	0.06	1.6	0.069	15.0	59.0	0.9964	3.3	0.46	9.4	5
7.3	0.65	0.0	1.2	0.065	15.0	21.0	0.9946	3.39	0.47	10.0	7
7.4	0.7	0.0	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
8.6	0.42	0.39	1.8	0.068	6.0	12.0	0.99516	3.35	0.69	11.7	8

4. 输出标签

- `quality`：整数评分，通常范围为 **0–10**
- 实际分布集中在 **3–8**

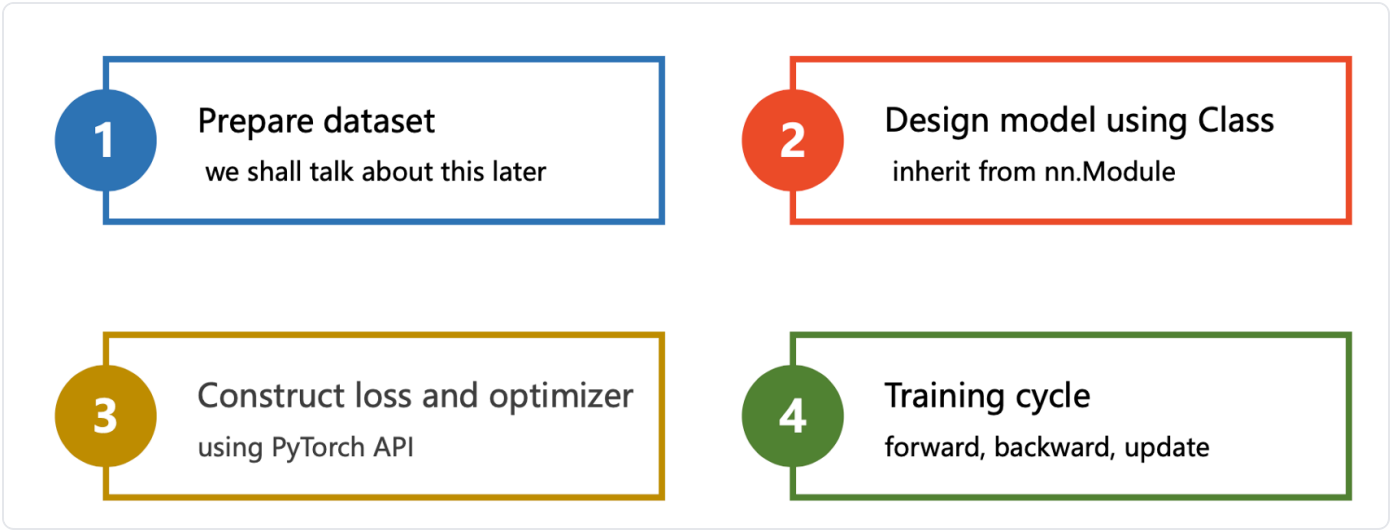
5. 任务类型

- **回归任务（Regression）**

6.评价指标

- MSE

7. 项目流程



1.2、数据集

1.数据集样本

固定酸度	挥发酸度	柠檬酸	残糖	氯化物	游离二氧化硫	总二氧化硫	密度	pH 值	硫酸盐	酒精度	品质
7.4	0.7	0.0	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
9.2	0.52	1.0	3.4	0.61	32.0	69.0	0.9996	2.74	2.0	9.4	4
7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.997	3.26	0.65	9.8	5
11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.998	3.16	0.58	9.8	6
10.4	0.61	0.49	2.1	0.2	5.0	16.0	0.9994	3.16	0.63	8.4	3
7.4	0.66	0.0	1.8	0.075	13.0	40.0	0.9978	3.51	0.56	9.4	5
7.9	0.6	0.06	1.6	0.069	15.0	59.0	0.9964	3.3	0.46	9.4	5
7.3	0.65	0.0	1.2	0.065	15.0	21.0	0.9946	3.39	0.47	10.0	7
7.4	0.7	0.0	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
8.6	0.42	0.39	1.8	0.068	6.0	12.0	0.99516	3.35	0.69	11.7	8

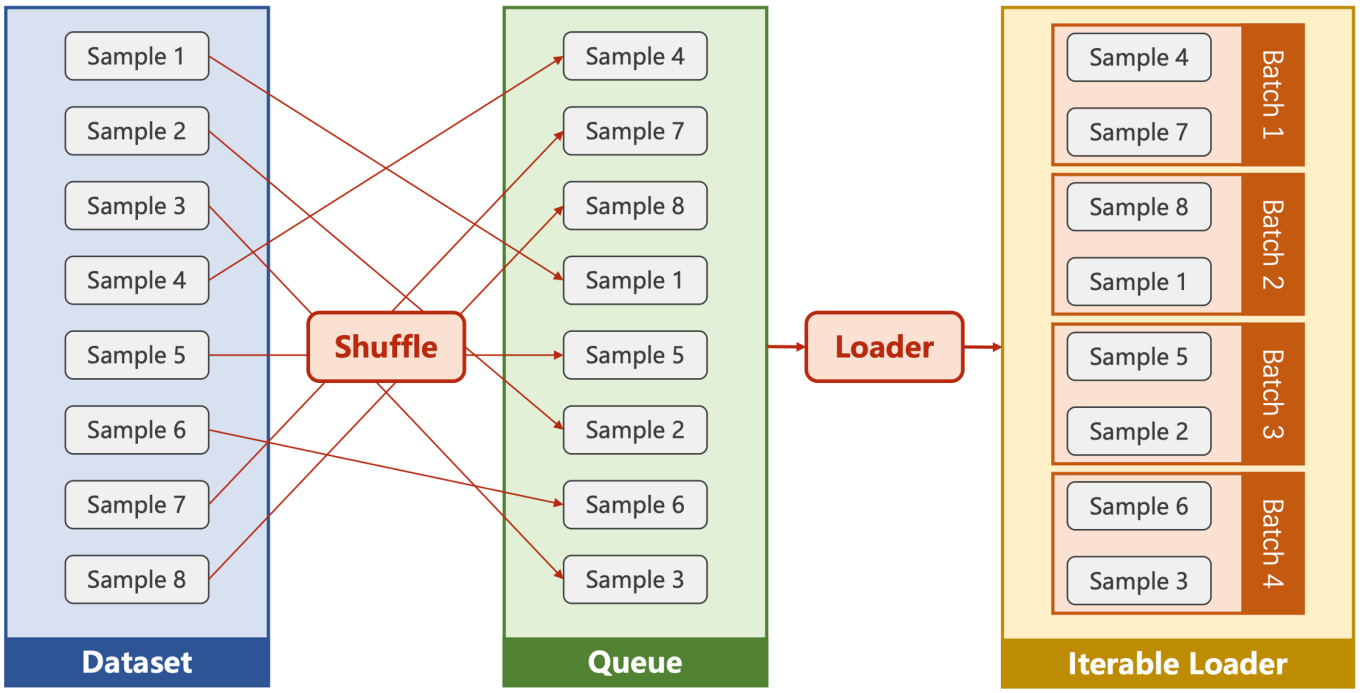
Sample

Feature

2.数据集加载

DataLoader 类 解释

官方 api地址：https://docs.pytorch.org/docs/stable/data.html




```
import torch
# Dataset 是 PyTorch 中用于“自定义数据集”的抽象基类
# 我们需要继承它，并实现其中的几个关键方法
from torch.utils.data import Dataset

# DataLoader 是 PyTorch 提供的数据加载器
# 负责：按 batch 取数据、打乱数据、多进程加载等
from torch.utils.data import DataLoader

# ============================
# 自定义数据集类 WineDataset
# ============================
# WineDataset 继承自 Dataset
# 用于封装“葡萄酒数据集”的读取逻辑
class WineDataset(Dataset):

    def __init__(self, filepath):
        """
        构造函数，在创建数据集对象时被调用一次

        参数：
        filepath：数据文件路径（例如 CSV 文件）
```

```
        通常在这里完成：
        1. 数据文件的读取
        2. 特征和标签的拆分
        3. 数据格式转换（numpy → torch.tensor）
        """
        pass  # 当前未实现，后续可在此处编写数据读取逻辑

    def __len__(self):
        """
        该魔法方法返回数据集的“样本总数”

        当你调用：
            len(dataset)
        或 DataLoader 需要知道数据集大小时，
        就会自动调用该方法

        返回值：
        int，表示数据集中一共有多少条样本
        """
        pass  # 例如：return self.len

    def __getitem__(self, index):
        """
        该魔法方法定义“如何根据索引 index 取出一条样本”

        当你调用：
            dataset[index]
        或 DataLoader 在内部取数据时，
        就会自动调用该方法

        参数：
        index：样本索引（0 ~ len(dataset)-1）

        返回值：
        一般返回 (feature, label)
        例如：
            return x, y
        """
        pass
```

```
# ============================
# 使用 DataLoader 加载数据
# ============================

# 创建数据集对象
# ⚠️ 注意：这里的 WineDataset() 实际上应该传入 filepath
# 例如：WineDataset("wine.csv")
dataset = WineDataset()

# 使用 DataLoader 对数据集进行封装
train_loader = DataLoader(
    dataset=dataset,   # 传入自定义的数据集对象
    batch_size=32,     # 每个 batch 中包含 32 个样本
    shuffle=True,      # 是否在每个 epoch 开始时打乱数据
    num_workers=2      # 使用 2 个子进程并行加载数据（加快 IO）
)

"""
DataLoader 的作用总结：
1. 自动调用 dataset.__getitem__() 取数据
2. 自动调用 dataset.__len__() 获取数据集大小
3. 将单条样本拼接成 batch
4. 支持打乱数据、并行加载
"""
```

3.具体实现

```
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader

# ============================================================
# 1. 数据集类（自定义 PyTorch Dataset）
# ============================================================
class WineDataset(Dataset):
    """
    WineDataset 用于读取葡萄酒质量数据集，并将其封装成
    PyTorch 可直接使用的 Dataset 格式。
```

```
    每一条数据包含：
    - 特征：前 11 列（连续数值特征）
    - 标签：最后 1 列（葡萄酒质量评分，回归任务）
    """
    def __init__(self, filepath):
        """
        初始化数据集
        :param filepath: CSV 文件路径
        """
        # 用 Python 列表暂存特征和标签，读取完成后再统一转 tensor
        features = []
        labels = []

        # 使用 with open 方式读取文件，避免文件未正确关闭
        with open(filepath, 'r', encoding='utf-8') as f:
            lines = f.readlines()

            # 跳过第一行（表头）
            for line in lines[1:]:
                # 去除每一行首尾的空白符（换行符、空格等）
                line = line.strip()

                # 原始 wine 数据集一般用分号 ; 分隔
                # 为了兼容不同格式，这里做一次判断
                if ';' in line:
                    parts = line.split(';')
                else:
                    parts = line.split(',')

                # 将字符串列表转换为 float32 的 numpy 数组
                data = np.array(parts, dtype=np.float32)

                # 前 11 列作为输入特征
                feature = data[:11]

                # 最后一列作为标签（回归值）
                # 使用 data[[-1]] 保证 shape 是 (1,)
                label = data[[-1]]

                features.append(feature)
```

```
                labels.append(label)

        # 将 Python list → numpy array → torch Tensor
        # x_data: [N, 11]
        # y_data: [N, 1]
        self.x_data = torch.from_numpy(np.array(features, dtype=np.float32))
        self.y_data = torch.from_numpy(np.array(labels, dtype=np.float32))

        # 数据集大小
        self.len = len(self.x_data)

    def __len__(self):
        """
        返回数据集样本数量
        DataLoader 内部会调用这个函数
        """
        return self.len

    def __getitem__(self, index):
        """
        根据索引 index 返回一条样本
        :return: (feature, label)
        """
        return self.x_data[index], self.y_data[index]

# ================================================================
# 数据加载器构建函数
# ================================================================
def get_dataset_loader(train_file_path, val_file_path, test_file_path, batch_size):
    """
    构建训练 / 验证 / 测试 DataLoader
    """
    # 分别加载训练、验证、测试数据集
    train_dataset = WineDataset(train_file_path)
    val_dataset = WineDataset(val_file_path)
    test_dataset = WineDataset(test_file_path)

    # 数据集大小统计
    train_size = len(train_dataset)
    val_size = len(val_dataset)
```

```
    test_size = len(test_dataset)

    print(f"训练集: {train_size} | 验证集: {val_size} | 测试集: {test_size}")

    # DataLoader:
    # - 训练集 shuffle=True（每个 epoch 打乱数据）
    # - 验证 / 测试集不打乱
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
    test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

    return train_loader, val_loader, test_loader

batch_size = 32
# 数据路径设置
train_file_path = 'wine_train.csv'
val_file_path = 'wine_val.csv'
test_file_path = 'wine_test.csv'

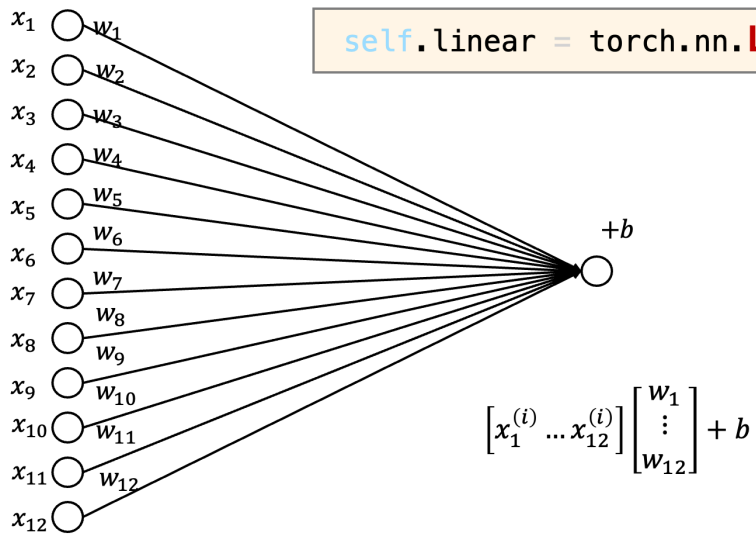
# 得到数据集
train_loader, val_loader, test_loader = get_dataset_loader(train_file_path, val_file_path, test

for i, data in enumerate(train_loader):
    x_features, labels = data
    import pdb;pdb.set_trace()
```

1.3、模型

1.nn.Linear()

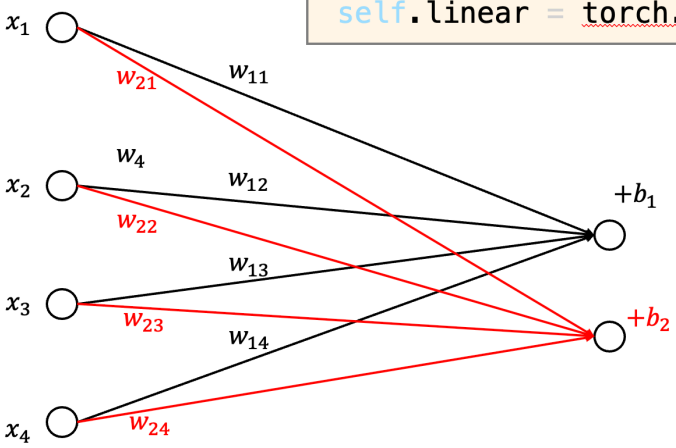
多输入逻辑回归模型




```
self.linear = torch.nn.Linear(12, 1)
```

$$\begin{bmatrix} x_1^{(i)} \dots x_{12}^{(i)} \end{bmatrix} \begin{bmatrix} w_1 \\ \vdots \\ w_{12} \end{bmatrix} + b$$

多输入逻辑回归模型




```
self.linear = torch.nn.Linear(4, 2)
```

$$[y_1\ y_2] = \begin{bmatrix} x_1^{(i)} x_2^{(i)} x_3^{(i)} x_4^{(i)} \end{bmatrix} \begin{bmatrix} w_{11} w_{21} \\ w_{12} w_{22} \\ w_{13} w_{23} \\ w_{14} w_{24} \end{bmatrix} + [b_1 b_2]$$

多输入逻辑回归模型

逻辑回归模型

$$\hat{y}^{(i)} = \sigma(x^{(i)} * \omega + b)$$

$$\sum_{n=1}^{12} x_n * w_n = \begin{bmatrix} x_1^{(i)} \dots x_{12}^{(i)} \end{bmatrix} \begin{bmatrix} w_1 \\ \vdots \\ w_{12} \end{bmatrix}$$

逻辑回归模型

$$\hat{y}^{(i)} = \sigma(\sum_{n=1}^{12} x_n * w_n + b)$$

逻辑回归模型

$$\hat{y}^{(i)} = \sigma\left(\begin{bmatrix} x_1^{(i)} \dots x_{12}^{(i)} \end{bmatrix} \begin{bmatrix} w_1 \\ \vdots \\ w_{12} \end{bmatrix} + b\right) = \sigma(z^{(i)})$$

计算过程

$$\begin{bmatrix} \hat{y}^{(1)} \\ \vdots \\ \hat{y}^{(12)} \end{bmatrix} = \begin{bmatrix} \sigma(z^{(1)}) \\ \vdots \\ \sigma(z^{(12)}) \end{bmatrix} = \sigma(\begin{bmatrix} z^{(1)} \\ \vdots \\ z^{(12)} \end{bmatrix})$$

Sigmoid function is in an element-wise fashion.

```
class Model(torch.nn.Module):
    def __init__(self):
        super(Model, self).__init__()
        self.linear = torch.nn.Linear(12, 1)
        self.sigmoid = torch.nn.Sigmoid()
    def forward(self, x):
        x = self.sigmoid(self.linear(x))
        return x
model = Model()
```

$$z^{(1)} = \begin{bmatrix} x_1^{(1)} & \dots & x_{12}^{(1)} \end{bmatrix} \begin{bmatrix} w_1 \\ \vdots \\ w_{12} \end{bmatrix} + b$$

$$\vdots$$

$$z^{(N)} = \begin{bmatrix} x_1^{(N)} & \dots & x_{12}^{(N)} \end{bmatrix} \begin{bmatrix} w_1 \\ \vdots \\ w_{12} \end{bmatrix} + b$$

$N \times 1$

$$\begin{bmatrix} z^{(1)} \\ \dots \\ z^{(N)} \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & \cdots & x_{12}^{(1)} \\ \vdots & \ddots & \vdots \\ x_1^{(N)} & \cdots & x_{12}^{(N)} \end{bmatrix} \begin{bmatrix} w_1 \\ \vdots \\ w_{12} \end{bmatrix} + \begin{bmatrix} b \\ \vdots \\ b \end{bmatrix}$$

$N \times 1$ $N \times 8$ $N \times 1$ $N \times 1$

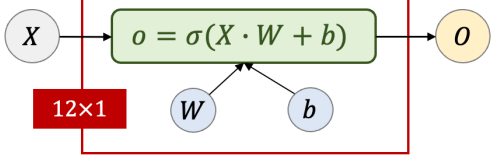
线性层

```
self.linear = torch.nn.Linear(12, 1)
```

每次输入的采样大小

$$X = \begin{bmatrix} x_1^{(1)} & \cdots & x_{12}^{(1)} \\ \vdots & \ddots & \vdots \\ x_1^{(N)} & \cdots & x_{12}^{(N)} \end{bmatrix}$$

线性层



每次采样的输出大小

$$O = \begin{bmatrix} o^{(1)} \\ \vdots \\ o^{(N)} \end{bmatrix}$$

$$O = \sigma\left(\begin{bmatrix} x_1^{(1)} & \cdots & x_{12}^{(1)} \\ \vdots & \ddots & \vdots \\ x_1^{(N)} & \cdots & x_{12}^{(N)} \end{bmatrix}\begin{bmatrix} w_1 \\ \vdots \\ w_{12} \end{bmatrix} + [b]\right)$$

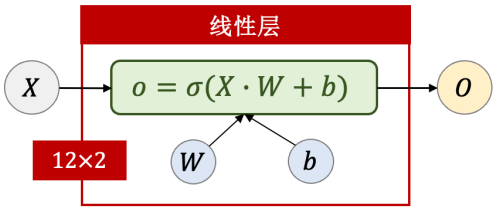
线性层

```
self.linear = torch.nn.Linear(12, 2)
```

每次输入的采样大小

$$X = \begin{bmatrix} x_1^{(1)} & \cdots & x_{12}^{(1)} \\ \vdots & \ddots & \vdots \\ x_1^{(N)} & \cdots & x_{12}^{(N)} \end{bmatrix}$$

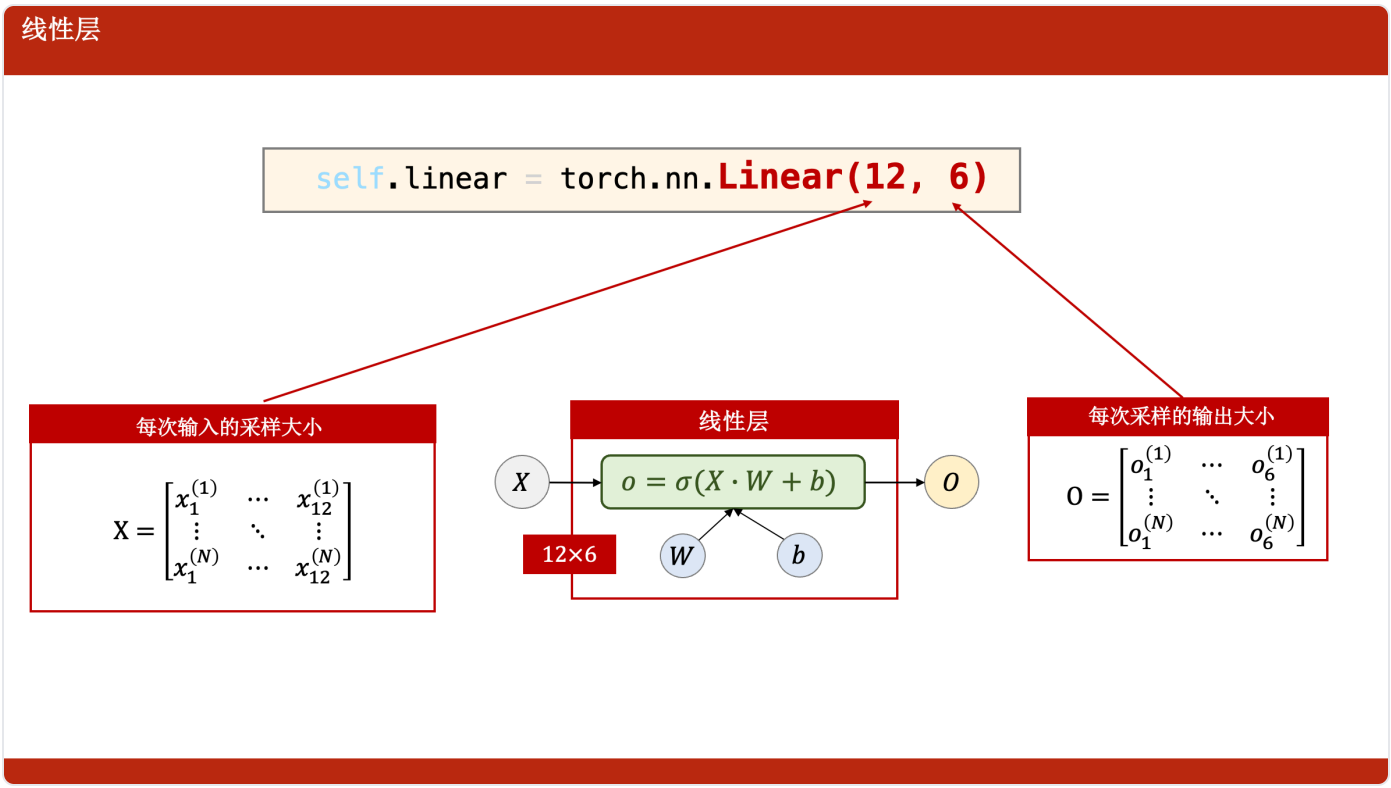
线性层



每次采样的输出大小

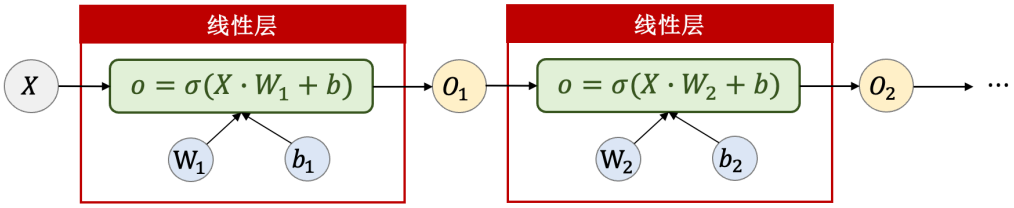
$$O = \begin{bmatrix} o_1^{(1)} & o_2^{(1)} \\ \vdots & \vdots \\ o_1^{(N)} & o_2^{(N)} \end{bmatrix}$$

$$X = \begin{bmatrix} x_1^{(1)} & \cdots & x_{12}^{(1)} \\ \vdots & \ddots & \vdots \\ x_1^{(N)} & \cdots & x_{12}^{(N)} \end{bmatrix}\begin{bmatrix} w_{11} & w_{21} \\ \vdots & \vdots \\ w_{112} & w_{212} \end{bmatrix} + [b_1 b_2]$$

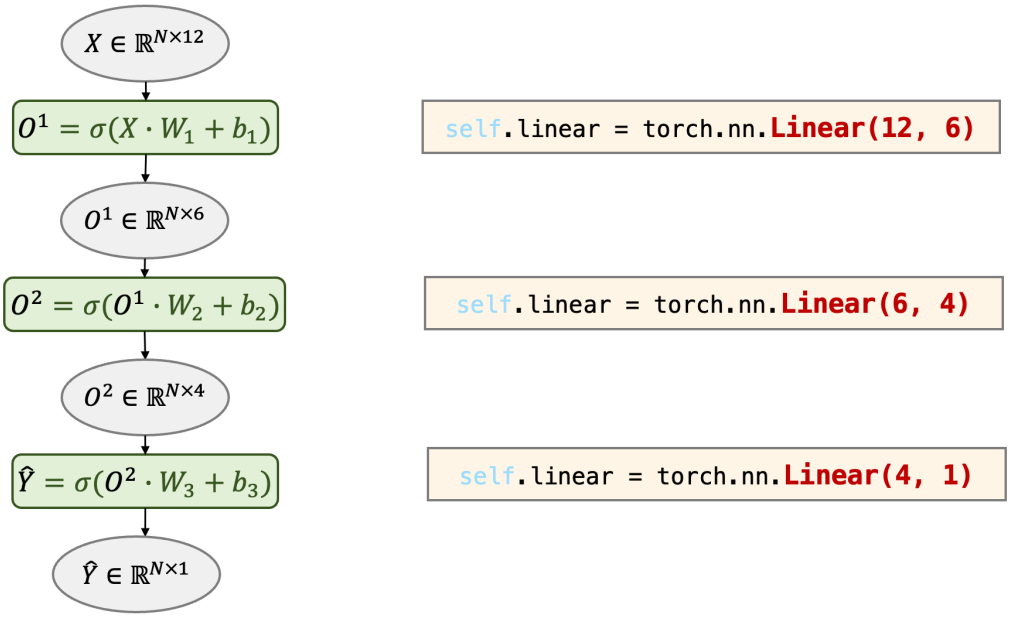

线性层
self.linear = torch.nn.Linear(12, 6)
每次输入的采样大小
$X = \begin{bmatrix} x_1^{(1)} & \cdots & x_{12}^{(1)} \\ \vdots & \ddots & \vdots \\ x_1^{(N)} & \cdots & x_{12}^{(N)} \end{bmatrix}$
线性层
X
$o = \sigma(X \cdot W + b)$
O
12×6
W
b
每次采样的输出大小
$O = \begin{bmatrix} o_1^{(1)} & \cdots & o_6^{(1)} \\ \vdots & \ddots & \vdots \\ o_1^{(N)} & \cdots & o_6^{(N)} \end{bmatrix}$

2.定义模型

神经网络



人工神经网络




```
# ============================================================
# 2. 模型定义（多层感知机 MLP）
# ============================================================
class WineQualityModel(torch.nn.Module):
```

```
    """
    用于葡萄酒质量预测的回归模型
    网络结构：
        11 → 6 → 4 → 1
    """
    def __init__(self, input_dim=11):
        super(WineQualityModel, self).__init__()

        # 三个全连接层
        self.linear1 = torch.nn.Linear(input_dim, 6)
        self.linear2 = torch.nn.Linear(6, 4)
        self.linear3 = torch.nn.Linear(4, 1)

        # 激活函数（ReLU）
        self.activation = torch.nn.ReLU()

    def forward(self, x):
        """
        前向传播
        """
        x = self.activation(self.linear1(x))
        x = self.activation(self.linear2(x))

        # 回归任务最后一层不加激活函数
        x = self.linear3(x)
        return x


model = WineQualityModel(input_dim=11)
```

1.4、损失函数和优化器

3、构建损失函数和优化器

MSE损失函数

$$loss = \frac{1}{N}\sum_{n=1}^{N}(y_n - \hat{y}_n)^2$$

参数更新

$$\omega = \omega - \alpha\frac{\partial loss}{\partial \omega}$$

```
criterion = torch.nn.MSELoss(reduction='mean')
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
```

```
criterion = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

1.5、模型训练

```
# ==================================================
# 训练循环（不封装，保持原始结构）
# ==================================================
for epoch in range(epochs):

    # 切换为训练模式
    model.train()

    # 遍历一个 epoch 中的所有 batch
    for inputs, labels in train_loader:
        # 梯度清零
        optimizer.zero_grad()

        # 前向传播
```

```
        outputs = model(inputs)

        # 计算损失
        loss = criterion(outputs, labels)

        # 反向传播
        loss.backward()

        # 参数更新
        optimizer.step()
```

1.6、完整代码

- 引入了模型权重保存、模型验证、模型测试、画曲线等。

```
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt

# ==============================================================
# 1. 数据集类（自定义 PyTorch Dataset）
# ==============================================================
class WineDataset(Dataset):
    """
    WineDataset 用于读取葡萄酒质量数据集，并将其封装成
    PyTorch 可直接使用的 Dataset 格式。

    每一条数据包含：
    - 特征：前 11 列（连续数值特征）
    - 标签：最后 1 列（葡萄酒质量评分，回归任务）
    """
    def __init__(self, filepath):
        """
        初始化数据集
        :param filepath: CSV 文件路径
        """
```

```
        # 用 Python 列表暂存特征和标签，读取完成后再统一转 tensor
        features = []
        labels = []

        # 使用 with open 方式读取文件，避免文件未正确关闭
        with open(filepath, 'r', encoding='utf-8') as f:
            lines = f.readlines()

            # 跳过第一行（表头）
            for line in lines[1:]:
                # 去除每一行首尾的空白符（换行符、空格等）
                line = line.strip()

                # 原始 wine 数据集一般用分号 ; 分隔
                # 为了兼容不同格式，这里做一次判断
                if ';' in line:
                    parts = line.split(';')
                else:
                    parts = line.split(',')

                # 将字符串列表转换为 float32 的 numpy 数组
                data = np.array(parts, dtype=np.float32)

                # 前 11 列作为输入特征
                feature = data[:11]

                # 最后一列作为标签（回归值）
                # 使用 data[[-1]] 保证 shape 是 (1,)
                label = data[[-1]]

                features.append(feature)
                labels.append(label)

        # 将 Python list → numpy array → torch Tensor
        # x_data: [N, 11]
        # y_data: [N, 1]
        self.x_data = torch.from_numpy(np.array(features, dtype=np.float32))
        self.y_data = torch.from_numpy(np.array(labels, dtype=np.float32))

        # 数据集大小
```

```
        self.len = len(self.x_data)

    def __len__(self):
        """
        返回数据集样本数量
        DataLoader 内部会调用这个函数
        """
        return self.len

    def __getitem__(self, index):
        """
        根据索引 index 返回一条样本
        :return: (feature, label)
        """
        return self.x_data[index], self.y_data[index]

# ================================================================
# 数据加载器构建函数
# ================================================================
def get_dataset_loader(train_file_path, val_file_path, test_file_path, batch_size):
    """
    构建训练 / 验证 / 测试 DataLoader
    """
    # 分别加载训练、验证、测试数据集
    train_dataset = WineDataset(train_file_path)
    val_dataset = WineDataset(val_file_path)
    test_dataset = WineDataset(test_file_path)

    # 数据集大小统计
    train_size = len(train_dataset)
    val_size = len(val_dataset)
    test_size = len(test_dataset)

    print(f"训练集: {train_size} | 验证集: {val_size} | 测试集: {test_size}")

    # DataLoader:
    # - 训练集 shuffle=True（每个 epoch 打乱数据）
    # - 验证 / 测试集不打乱
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
```

```
    test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

    return train_loader, val_loader, test_loader

# ============================================================
# 2. 模型定义（多层感知机 MLP）
# ============================================================
class WineQualityModel(torch.nn.Module):
    """
    用于葡萄酒质量预测的回归模型
    网络结构：
        11 → 6 → 4 → 1
    """
    def __init__(self, input_dim=11):
        super(WineQualityModel, self).__init__()

        # 三个全连接层
        self.linear1 = torch.nn.Linear(input_dim, 6)
        self.linear2 = torch.nn.Linear(6, 4)
        self.linear3 = torch.nn.Linear(4, 1)

        # 激活函数（ReLU）
        self.activation = torch.nn.ReLU()

    def forward(self, x):
        """
        前向传播
        """
        x = self.activation(self.linear1(x))
        x = self.activation(self.linear2(x))

        # 回归任务最后一层不加激活函数
        x = self.linear3(x)
        return x

# ============================================================
# 3. 验证 / 测试评估函数
# ============================================================
def evaluate(model, data_loader, criterion, device):
    """
```

```
    在验证集或测试集上评估模型
    """
    # 切换到评估模式（影响 Dropout / BatchNorm）
    model.eval()

    total_loss = 0
    predictions = []
    actuals = []

    # 评估阶段不计算梯度，节省显存和计算
    with torch.no_grad():
        for inputs, labels in data_loader:
            inputs = inputs.to(device)
            labels = labels.to(device)

            # 前向传播
            outputs = model(inputs)

            # 计算损失
            loss = criterion(outputs, labels)
            total_loss += loss.item()

            # 保存预测值和真实值（用于分析）
            predictions.extend(outputs.cpu().numpy())
            actuals.extend(labels.cpu().numpy())

    # 平均 loss
    avg_loss = total_loss / len(data_loader)
    return avg_loss, predictions, actuals

# ============================================================
# 4. 主流程
# ============================================================
def main():
    # ---------------- 超参数 ----------------
    epochs = 100
    learning_rate = 0.001
    batch_size = 128

    # ---------------- 数据路径 ----------------
```

```
train_file_path = 'wine_train.csv'
val_file_path = 'wine_val.csv'
test_file_path = 'wine_test.csv'

# ---------------- 数据加载 ----------------
train_loader, val_loader, test_loader = get_dataset_loader(
    train_file_path,
    val_file_path,
    test_file_path,
    batch_size
)

# ---------------- 设备选择 ----------------
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"使用设备: {device}")

# ---------------- 模型 / 损失 / 优化器 ----------------
model = WineQualityModel(input_dim=11).to(device)

# 回归任务常用 MSELoss
criterion = torch.nn.MSELoss()

# 使用 SGD 优化器
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

# ---------------- 训练准备 ----------------
best_val_loss = float('inf')

print("\n开始训练...")
print("=" * 50)

train_losses = []
val_losses = []

# ======================================================
# 训练循环（不封装，保持原始结构）
# ======================================================
for epoch in range(epochs):

    # 切换为训练模式
```

```
        model.train()
        total_loss = 0

        # 遍历一个 epoch 中的所有 batch
        for inputs, labels in train_loader:
            inputs = inputs.to(device)
            labels = labels.to(device)

            # 梯度清零
            optimizer.zero_grad()

            # 前向传播
            outputs = model(inputs)

            # 计算损失
            loss = criterion(outputs, labels)

            # 反向传播
            loss.backward()

            # 参数更新
            optimizer.step()

            total_loss += loss.item()

        # 训练集平均 loss
        train_loss = total_loss / len(train_loader)

        # ---------------- 验证 ----------------
        val_loss, _, _ = evaluate(model, val_loader, criterion, device)

        train_losses.append(train_loss)
        val_losses.append(val_loss)

        # 日志输出
        print(f"Epoch [{epoch+1:3d}/{epochs}] | "
              f"Train Loss: {train_loss:.4f} | "
              f"Val Loss: {val_loss:.4f} | "
              )
```

```
        # ---------------- 早停 / 保存最优模型 ----------------
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            torch.save(model.state_dict(), 'best_model.pth')

# ======================================================
# 5. Loss 曲线绘制
# ======================================================
plt.figure(figsize=(8, 5))
plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Validation Loss')

plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('Training & Validation Loss Curve')
plt.legend()
plt.grid(True)

# 保存图片到当前目录
plt.savefig('loss_curve.png', dpi=300, bbox_inches='tight')
# plt.show()

# ======================================================
# 6. 测试集评估
# ======================================================
print("\n" + "=" * 50)
print("在测试集上评估最佳模型...")

# 加载验证集上最优的模型参数
model.load_state_dict(torch.load('best_model.pth'))

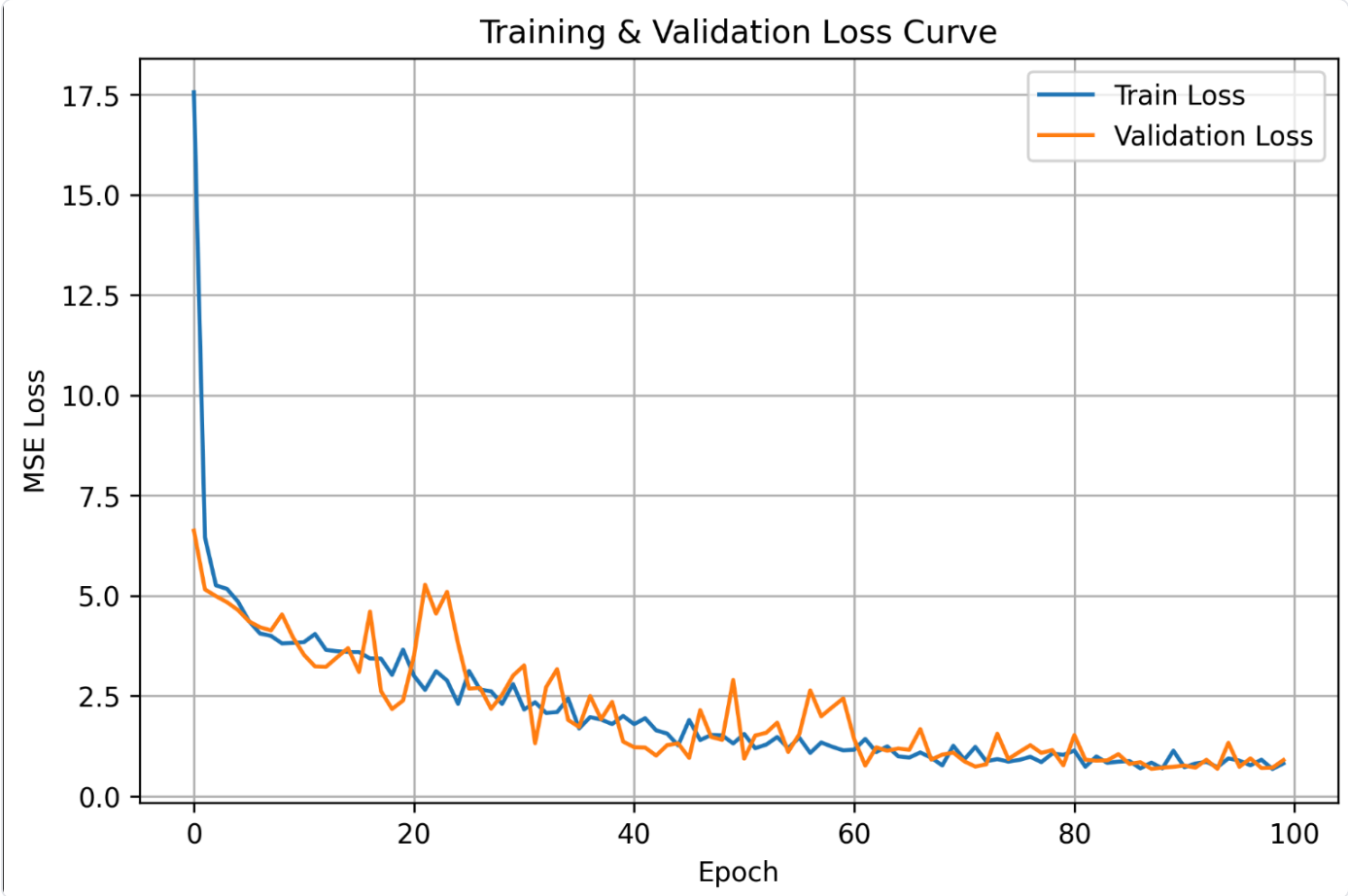
# 测试
test_loss, predictions, actuals = evaluate(
    model, test_loader, criterion, device
)

print(f"测试集 MSE Loss: {test_loss:.4f}")

predictions = np.array(predictions).flatten()
actuals = np.array(actuals).flatten()
```

```
    # 输出前 5 个样本的预测结果对比
    print("\n预测 vs 实际 (前5个样本):")
    for i in range(10):
        print(f"  预测: {predictions[i]:6.2f} | 实际: {actuals[i]:6.2f}")

# ============================================================
# 程序入口
# ============================================================
if __name__ == '__main__':
    main()
```



1.7、超参数调整及优化

1.超参数调整

- 模型深度

- 增加/减少 nn.Linear个数 -

- 模型宽度

- nn.Linear变宽/变窄 - base 测试集 MSE Loss: 0.6024 - 测试集 MSE Loss: 0.5931

```
class WineQualityModel(torch.nn.Module):
    """
    用于葡萄酒质量预测的回归模型
    网络结构:
        11 → 6 → 4 → 1
    """
    def __init__(self, input_dim=11):
        super(WineQualityModel, self).__init__()

        # 三个全连接层
        self.linear1 = torch.nn.Linear(input_dim, 32)
        self.linear2 = torch.nn.Linear(32, 24)
        self.linear3 = torch.nn.Linear(24, 1)

        # 激活函数（ReLU）
        self.activation = torch.nn.ReLU()

    def forward(self, x):
        """
        前向传播
        """
        x = self.activation(self.linear1(x))
        x = self.activation(self.linear2(x))

        # 回归任务最后一层不加激活函数
        x = self.linear3(x)
        return x
```

- 学习率

- 原始：learning_rate = 0.001 - 也没找到更好的

- Batch size

- 也没找到更好的

- 激活函数

```
# self.activation = torch.nn.ReLU()
# 测试集 MSE Loss: 0.5846
# self.activation = torch.nn.Tanh()
# 测试集 MSE Loss: 0.7286
# self.activation = torch.nn.ReLU6()
# 测试集 MSE Loss: 0.6522
self.activation = torch.nn.PReLU()
# 测试集 MSE Loss: 0.5660
```

- 训练轮数

- 100: 测试集 MSE Loss: 0.5660 - 1000: 测试集 MSE Loss: 0.5444

2.优化方法

- 更换不同优化器

```
    # https://docs.pytorch.org/docs/stable/generated/torch.optim.SGD.html
    # 使用 SGD 优化器
    # optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
    # 测试集 MSE Loss: 0.5519
    # optimizer = torch.optim.SGD(model.parameters(), momentum = 0.9, lr=learning_rate)
    # # 测试集 MSE Loss: 0.5353
    # optimizer = torch.optim.Adagrad(model.parameters(), lr=learning_rate)
    # 测试集 MSE Loss: 4.2505
    # optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    # # 测试集 MSE Loss: 0.5116
    optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)
    # # 测试集 MSE Loss: 0.5078
```

- L2正则化

```
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate, weight_decay=1e-4)
```

- Dropout

- 原来： 测试集 MSE Loss: 0.5257

```
        self.dropout = torch.nn.Dropout(p=0.1)


    def forward(self, x):
        x = self.activation(self.linear1(x))
        x = self.dropout(x)

        x = self.activation(self.linear2(x))
        x = self.dropout(x)

        x = self.linear3(x)
        return x

测试集 MSE Loss: 0.5810
```

- 早停机制

- 测试集 MSE Loss: 0.5047

```
patience = 20
patience_counter = 0

        # ---------------- 早停 / 保存最优模型 ----------------
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            torch.save(model.state_dict(), 'best_model.pth')
            patience_counter = 0   # 有提升，计数器清零
        else:
```

```
            patience_counter += 1  # 没提升，计数器 +1

        # 触发早停
        if patience_counter >= patience:
            print(f"Early stopping at epoch {epoch+1}")
            break
```

1.8、更换数据集

kaggle挑战赛波士顿房价预测数据集为例，https://www.kaggle.com/competitions/dl-bhpp/overview

波士顿房价预测

Overview Data Code Models Discussion Leaderboard Rules Team Submissions

数据介绍

此数据是一份源于美国某经济学杂志上，分析研究波士顿房价(Boston House Price)的数据集。数据集中的每一行数据都是对波士顿周边或城镇房价的描述：

- CRIM: 城镇人均犯罪率
- ZN: 住宅用地所占比例
- INDUS: 城镇中非住宅用地所占比例
- CHAS: CHAS 虚拟变量,用于回归分析
- NOX: 环保指数
- RM: 每栋住宅的房间数
- AGE: 1940 年以前建成的自住单位的比例
- DDIS: 距离 5 个波士顿的就业中心的加权距离。
- RAD: 距离高速公路的便利指数
- TAX: 每一万美元的不动产税率
- PRTATIO: 城镇中的教师学生比例
- B: 城镇中的黑人比例
- LSTAT: 地区中有多少房东属于低收入人群
- MEDV: 自住房屋房价中位数（也就是均价）

数据集合

数据集由train.csv、test.scv和saample.csv三个文件组成，其中train.csv是训练数据集，共有404条数据，包含13列特征和一列房价；test.csv为测试数据集，共有102条数据，包含13列特征，无房价信息。sample.csv文件是提交结果时参考的模板文件，提交结果按照此文件格式提交，格式错误会影响分数。

- 下载数据集并且划分

```
cd dl-bhpp/
python3  split_dataset.py
```

- 模型训练

```
python3 main_new_dataset.py
```

- 测试提交

- 将产生的submission.csv 提交到kaggle上，得出分数和排名。

二、简历怎么写?

复制地址：基于深度学习的葡萄酒质量评分实践(可下载复制)

基于深度学习的葡萄酒质量预测

- 任务：基于葡萄酒的 11 维理化检测指标（如酸度、pH、酒精度等），利用 UCI Machine Learning Repository 公共 Wine Quality 数据集（白葡萄酒 4898 条样本），构建监督学习回归模型预测葡萄酒质量评分
- 技术方案：使用多层感知机（MLP）对 11 维理化特征进行建模，mse作为损失函数，并尝试不同网络深度、宽度、激活函数、学习率和优化器等超参数。
- 优化：针对模型过拟合问题，引入 **Dropout、L2 正则化与 Early Stopping** 提升泛化性能。
- 结果：在测试集上将 **MSE 从 0.6024 优化至 0.5078。**

三、面经

0、背景知识

1、简单介绍一下这个项目。

本项目基于葡萄酒的 11 维理化检测指标，构建监督学习回归模型，用于预测葡萄酒的质量评分。项目使用 PyTorch 深度学习框架，以 UCI Machine Learning Repository 提供的白葡萄酒数据集（4898 条样本）为数据来源。通过对网络结构、训练超参数和正则化方法进行系统对比与调优，逐步提升模型在测试集上的预测性能与泛化能力。

2、该任务属于哪一类机器学习问题？为什么?

该任务属于监督学习中的回归问题。因为每个样本都包含明确的质量评分标签，模型需要学习从连续输入特征到连续输出数值之间的映射关系。

3、项目中代码量大约是多少?

项目代码总量约为 500 行左右，主要包括数据加载、模型定义、训练流程、评估函数和结果可视化等模块。

一、数据

1、项目中使用了什么数据集？数据来源与规模如何?

本项目使用的是公开数据集——UCI Machine Learning Repository 提供的 Wine Quality 数据集中的白葡萄酒子集，可通过 Kaggle 下载获得。 数据集中共包含 4898 条白葡萄酒样本，每条样本包含 11 个理化特征（均为浮点数）和 1 个质量评分标签。数据按 **7:1.5:1.5** 的比例随机划分为训练集、验证集和测试集，分别包含 **3446 条（约 70.4%）**、**752 条（约 15.4%）** 和 **700 条（约 14.3%）** 样本。

2、数据特征包含哪些内容？是否进行了预处理?

每个样本包含 11 个连续型特征，如固定酸度、挥发性酸度、柠檬酸、残糖、酒精度等，均为浮点数。 由于特征量纲不同，在建模前对所有特征进行了标准化处理。数据集中不存在缺失值，使用平均值代替。

3、标签的范围与分布情况如何?

标签为葡萄酒质量评分，取值范围为 3～9。样本主要集中在 5、6、7 分区间，其中 6 分样本最多（2198 条），其次为 5 分（1457 条）和 7 分（880 条）。低分样本较少，如 3 分仅 20 条、4 分 163 条；高分样本也较少，如 8 分 175 条、9 分仅 5 条，整体分布存在明显不均衡现象。

4、在 PyTorch 中加载数据集的一般流程是什么?

一般流程可概括为两步： 1）通过继承 `Dataset` 类实现 `getitem` 和 `len` 方法，用于按索引获取单条样本； 2）使用 `DataLoader` 对数据进行随机打乱并组成批次（batch），在训练过程中按批迭代加载数据用于模型训练与评估。

5、继承 Dataset 实现自定义数据集时需要实现哪三个函数?

需要实现三个函数：

- `init()`：用于初始化数据集和读取数据；
- `len()`：用于返回数据集的样本总数；
- `getitem()`：用于根据索引返回单个样本及其对应标签。

二、网络结构

1、项目中使用了什么模型结构？为什么选择这种结构?

项目采用多层感知机（MLP）作为主要模型结构。该结构适用于处理结构化表格数据，能够通过多层全连接和非线性变换刻画输入特征与输出目标之间的复杂关系。同时，MLP 结构简单、可控性强，便于系统比较不同网络配置对模型性能的影响。

2、网络的输入层、隐藏层和输出层如何设计?

输入层接收每个样本的 11 维特征向量；隐藏层由多层全连接层组成，并在每层后引入非线性激活函数以增强模型表达能力；输出层采用单个神经元并使用线性输出形式，用于预测连续数值形式的质量评分。

3、nn.Linear(4, 2) 表示什么？在带偏置的情况下有多少个可学习参数?

`nn.Linear(4, 2)` 表示一个输入维度为 4、输出维度为 2 的全连接层，其线性变换形式为 (y = Wx + b)。 在带偏置的情况下，权重参数数量为 ($4 \times 2 = 8$) 个，偏置参数为 2 个，总计 10 个可学习参数。

4、在 PyTorch 中定义一个模型通常需要实现哪两个函数?

通常需要实现：

- `init()`： 用于定义网络层结构和可学习参数；
- `forward()`： 用于定义模型的前向传播过程，即数据如何从输入经过各层变换得到输出。

三、损失函数

1、本项目使用了什么损失函数？为什么选择它?

项目使用均方误差（MSE）作为损失函数。MSE 能够衡量预测值与真实值之间的平均平方偏差，对较大误差给予更高惩罚，适用于回归任务中对预测精度要求较高的场景，同时具有良好的数学性质，便于梯度计算与优化。

四、优化器

1、项目中使用了哪些优化器？哪一个效果更好？为什么?

项目中对比了 SGD、带动量的 SGD、Adagrad 和 Adam 等优化器。实验结果表明，Adam 在该任务上取得了最优性能（测试集 MSE 最低）。 这是因为 Adam 同时结合了动量机制和自适应学

习率调整，能够在不同特征尺度下实现更稳定、更高效的参数更新，从而在收敛速度和最终效果上优于其他优化器。

五、训练超参数

1、项目中调节了哪些训练超参数?

调节的关键训练超参数包括学习率、batch size、网络层数与宽度、激活函数类型、dropout 比例、L2 正则化系数以及训练轮数等。

2、效果最好的超参数是什么?

在多组超参数与模型结构对比实验中，当前代码配置取得了最优测试性能，其主要超参数如下：

- 网络结构：多层感知机（MLP），结构为 11 → 32 → 24 → 1，中间层使用 PReLU 激活函数；
- 损失函数：均方误差损失（MSELoss）；
- 优化器：AdamW；
- 学习率：0.0001；
- 批大小（Batch Size）：128；
- 训练轮数与早停策略：最大训练轮数 1000，patience = 20，当验证集损失连续 20 个 epoch 未提升时提前停止训练。

在上述配置下，模型在测试集上取得了约 MSE = 0.5078 的性能。

六、评价指标

1、本项目使用什么评价指标评估模型性能？为什么？

项目主要使用测试集上的均方误差（MSE）作为评价指标，用于衡量模型预测值与真实评分之间的平均误差水平。

MSE 对较大预测误差较为敏感，能够较好反映模型在回归任务中的预测精度，适合用于对质量评分预测误差进行量化评估。

七、训练结果与优化

1、项目中遇到的主要问题是什么？是如何判断的？

在初始训练阶段，模型在训练集上的误差明显低于验证集和测试集，说明模型对训练数据拟合较好，但在新数据上的表现下降，存在过拟合现象。通过对比训练损失与验证损失曲线，可以直观判断模型泛化能力不足。

2、为缓解过拟合采取了哪些优化措施？这些措施产生了什么影响？

为缓解过拟合，引入了 Dropout、L2 正则化和 Early Stopping 等策略。Dropout 通过随机屏蔽部分神经元减少特征间依赖；L2 正则化通过限制参数规模降低模型复杂度；Early Stopping 通过在验证集性能不再提升时提前终止训练，防止模型继续拟合训练数据中的噪声。

通过上述优化策略，模型在测试集上的 MSE 从 0.6024 降低至 0.5078，说明模型泛化性能得到明显提升。

3、什么是 L2 正则化？它在模型训练中起什么作用？

L2 正则化是在损失函数中加入权重平方和惩罚项，用于限制模型参数整体规模。其作用是抑制参数过大，降低模型复杂度，从而减少对训练数据中噪声的过度拟合，提升模型的泛化能力。

4、什么是 Dropout？它在训练过程中如何发挥正则化作用？

Dropout 是一种随机正则化方法，在训练过程中以一定概率随机屏蔽部分神经元，使其在当前迭代中不参与前向和反向传播。该机制减少神经元之间的协同依赖，使模型在训练过程中相当于学习多个不同子网络，从而降低过拟合风险。

5、如果进一步改进该项目，可以从哪些方面入手？

在数据层面，可以引入更多样本，或将红葡萄酒与白葡萄酒数据联合建模，以增强模型的泛化能力；同时针对标签分布不均衡问题，可考虑采用重采样、加权损失函数等方法进行优化。 在模型层面，可以尝试更深的网络结构。 在训练策略层面，可以进一步优化学习率调度策略，以提升训练稳定性和泛化性能。